SECTION 1: Business and Data Visualisation Context

Introduction

YourFirm.de is one of Germany's online leading job board that focuses in small and medium-sized enterprises (SMEs), known in German as the *Mittelstand*. Yourfirm is a public company which operates in the delivery services through an online platform offering a job opportunities for professionals, managers, and employers in the mid-sized companies.

This notebook examines a dataset gathered from the YourFirm website in March 2026. Each data holds information about the job title, hiring company, geographic location (city, state, country, and GPS coordinates), job categories, and posting dates.

Our aim is to make an explanatory data analysis pipeline that analyzes data to generate meaningful insights. Accordingly, we focus on answering questions such as:

1. Where are jobs concentrated geographically?
2. Which cities have the most listings, and are there underserved regions?
3. Which companies are hiring?
4. Which companies maintain the highest market share on the platform, and How concentrated is the market?
5. Which job categories are in demand?
6. How is the market globally recognized? What proportion of job titles are presented in English and German?
7. When do recruiters post?

The audience for this analysis is the **management team at YourFirm.de**, who can use these insights to guide marketing strategy, identify growth opportunities, and improve the platform experience for both employers and job seekers.

What do we want our audience to know or do?

We want the leadership team to understand the **structure and trends** hidden in their own platform data. Accordingly, we want the leadership team to do the following things:

- Point out which geographic regions are well-served and which are underprivileged, so they can target expansion efforts.
- Recognize which companies and industries stand over their platform,and to determine client concentration risk.
- Analyze hiring trends such as the enlarging usage of English language in job titles.
- Leverage these analytical findings to optimize data-driven decisions across marketing channels, employer partnerships, and platform utilities.

What kind of data do we have?

The Yourfirm.de dataset contains job listings posted in March 2026. Each record has 19 columns including:

- **Job information:** title, description (position_addition), categories
- **Company information:** company name, contact details
- **Location:** region/city, state, country, postal code, GPS coordinates
- **Timing:** posting start date, scrape timestamp

Source: https://www.yourfirm.de/

Github Link : https://github.com/sifen2022-cpu/CSV-file.git

Name: Sifen Dagne Semu

ID: GH1029686

SECTION 2: Data Exploration Discussion

2. Data Exploration

In this section, we import the source dataset and analyze its characteristics prior to data prepocessing. Our goal is to understand the data distribution, identify quality issues, and discover what curation is needed.

In [ ]:
```
import pandas as pd
import plotly.express as px
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
```

```
import matplotlib.patches as mpatches
import numpy as np
import squarify
import re
from collections import Counter
```

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", forc
e_remount=True).
```

In [ ]:
```
import pandas as pd
file_path = '/content/drive/MyDrive/yourfirm-jobs.csv'
df = pd.read_csv(file_path)
df.head()
```

Out[ ]:

	source_id	reference_number	company_name	title	position_addition	source_url	
0	YF28699155	YF-38434	Schattdecor SE	Leitung Druckformfertigung (m/w/d)	View job here Vollzeit Thansau, 83101 Rohrdorf...	https://www.yourfirm.de/job/schattdecor-se/lei...	https
1	YF28699133	YF-38348	Bergader Privatkäserei GmbH	Milchwirtschaftlicher Laborant (m/w/d)	Entnahme von Proben von Rohwaren, Hilfs- und Z...	https://www.yourfirm.de/job/bergader-privatkae...	https
2	YF28752016	YF-39312	Praxis Dr. med. Dirk Herting	MFA / Medizinische Fachangestellte (m/w/d) Tei...	Abgeschlossene Ausbildung als medizinische Fac...	https://www.yourfirm.de/job/praxis-dr-med-dirk...	
3	YF28751901	YF-39181	Spaichinger Nudelmacher GmbH	Mitarbeiter im Verkauf (m/w/d) Minijob oder Te...	Die Spaichinger Nudelmacher GmbH ist eine Toch...	https://www.yourfirm.de/job/spaichinger-nudelm...	
4	YF28751903	YF-39182	Spaichinger Nudelmacher GmbH	Industrieelektriker (m/w/d)	Die Spaichinger Nudelmacher GmbH ist eine Toch...	https://www.yourfirm.de/job/spaichinger-nudelm...	

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 47750 entries, 0 to 47749
Data columns (total 19 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   source_id          47750 non-null  object
 1   reference_number   1965 non-null   object
 2   company_name       47750 non-null  object
 3   title              47750 non-null  object
 4   position_addition  47143 non-null  object
 5   source_url         2433 non-null   object
 6   application_url    47750 non-null  object
 7   contact_email      47750 non-null  object
 8   contact_phone      0 non-null      float64
 9   region             47750 non-null  object
 10  postal_code        44897 non-null  float64
 11  state              44909 non-null  object
 12  country            45711 non-null  object
 13  latitude           44905 non-null  float64
 14  longitude          44905 non-null  float64
 15  categories         36653 non-null  object
 16  subcategories      36653 non-null  object
 17  start_date         47750 non-null  object
 18  scraped_at         47750 non-null  object
dtypes: float64(4), object(15)
memory usage: 6.9+ MB
```

In [ ]:
```
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(1)
missing_df = pd.DataFrame({
    'Missing Count': missing,
    'Missing %': missing_pct
})
missing_df = missing_df.sort_values('Missing %', ascending=False)
print("=== Missing Values Per Column ===\n")
print(missing_df.to_string())
```

```
=== Missing Values Per Column ===

                   Missing Count  Missing %
contact_phone              47750      100.0
reference_number           45785       95.9
source_url                 45317       94.9
categories                 11097       23.2
subcategories              11097       23.2
latitude                    2845        6.0
longitude                   2845        6.0
postal_code                 2853        6.0
state                       2841        5.9
country                     2039        4.3
position_addition            607        1.3
company_name                   0        0.0
source_id                      0        0.0
title                          0        0.0
application_url                0        0.0
region                         0        0.0
contact_email                  0        0.0
start_date                     0        0.0
scraped_at                     0        0.0
```

Observation: The output reveals several important things:

- There are **4 numeric columns** (`float64`): `contact_phone` , `postal_code` , `latitude` , and `longitude` . The rest are text (`object`).
- `contact_phone` has **0 non-null values** — it is totally empty and does not have an effect for our analysis.
- `reference_number` has only **1,965 non-null values** out of 47,750 (~4%), making it nearly entirely empty.
- `source_url` has only **2,433 non-null values** (~5%).
- Geographic columns (`postal_code` , `state` , `latitude` , `longitude`) are missing for about **2,800–2,850 rows** (~6%).
- `categories` and `subcategories` are gone for about **11,100 rows** (~23%).

In [ ]:
```
print("Dataset Overview \n")
print(f"Total job postings:      {len(df):,}")
print(f"Unique companies:        {df['company_name'].nunique():,}")
print(f"Unique regions/cities:   {df['region'].nunique():,}")
print(f"Countries represented:   {df['country'].dropna().unique()}")
print(f"States covered:          {df['state'].dropna().nunique()}")
print(f"\nDate range (start_date):")
print(f"  From: {df['start_date'].min()}")
print(f"  To:   {df['start_date'].max()}")
```

```
Dataset Overview

Total job postings:      47,750
Unique companies:        13,665
Unique regions/cities:   4,564
Countries represented:   ['Deutschland' 'Österreich']
States covered:          24

Date range (start_date):
  From: 2026-02-11T23:00:00.000Z
  To:   2026-03-13T23:00:00.000Z
```

Observation: The overview reveals the scale and scope of the data:

- From **13,665 unique companies,** we have got **47,750 job postings,** which indicates the average company has about 3.5% postings, showing a platform small employers have influence than the large ones.
- Based on our geographic reporting, **4,564 unique regions/cities** cover the German-speaking market.
- The **two countries;** Germany and Austria, encompassing **24 states/federal regions** across both countries (16 German Bundesländer + 8 Austrian Bundesländer).
- **Date range:** February 11 to March 13, 2026 — roughly one month of listings. This is sufficient for analyzing daily and weekly patterns but not for assessing seasonal or year-over-year trends.

SECTION 3: Data Preprocessing

Now that we understand our dataset having 47,750 job posting rows, 19 columns, with significant missing values in several fields, duplicate records, and non-informative coloumns. Data cleaning is required for the dataset before visualization. The assessment brief asks

1. **Remove Duplicate** - We can remove duplicates using source ID that are not useful.
2. **Drop non-informative columns** — We can remove columns that are 95–100% empty.
3. **Eliminate low value columns** — We can remove `categories` and `subcategories` , `contact_phone` , `application_url` , `contact_email` , `scraped_at` , `source_url` which holds the data we dont need for our analysis.
4. **Address missing values** — We fill geographic NaNs with 'Unknown'

5. **Convert date columns** — We render string dates into datetime objects

In [ ]:
```
import pandas as pd
file_path = '/content/drive/MyDrive/yourfirm-jobs.csv'
df = pd.read_csv(file_path)

# Remove Duplicates (if any)
before = len(df)
df.drop_duplicates(subset='source_id', keep='first', inplace=True)
after = len(df)

print(f"Rows before: {before:,}")
print(f"Rows after:  {after:,}")
print(f"Duplicates removed: {before - after:,}")
```

```
Rows before: 47,750
Rows after:  40,986
Duplicates removed: 6,764
```

In [ ]:
```
# Drop Useless and Unhelpful Columns

cols_before = df.shape[1]
drop_cols = ['contact_phone', 'source_id', 'application_url', 'contact_email', 'scraped_at', 'reference_number'
df.drop(columns=drop_cols, inplace=True)

print(f"Dropped {len(drop_cols)} columns: {drop_cols}")
print(f"Columns reduced from {cols_before} → {df.shape[1]}")
print(f"\nRemaining columns:\n{list(df.columns)}")
```

```
Dropped 9 columns: ['contact_phone', 'source_id', 'application_url', 'contact_email', 'scraped_at', 'reference_n
umber', 'source_url', 'categories', 'subcategories']
Columns reduced from 19 → 10

Remaining columns:
['company_name', 'title', 'position_addition', 'region', 'postal_code', 'state', 'country', 'latitude', 'longitu
de', 'start_date']
```

In [ ]:
```
# Handle Missing Values in Country and State

print("Before filling:\n")
print(f"  country — missing: {df['country'].isnull().sum():,} rows")
print(f"  state   — missing: {df['state'].isnull().sum():,} rows")

df['country'] = df['country'].fillna('Unknown')
df['state'] = df['state'].fillna('Unknown')

print(f"\nAfter filling with 'Unknown':\n")
print(df['country'].value_counts().to_string())
print(f"\n  state 'Unknown' count: {(df['state'] == 'Unknown').sum():,}")
```

```
Before filling:

  country — missing: 1,646 rows
  state   — missing: 2,448 rows

After filling with 'Unknown':

country
Deutschland    35213
Österreich      4127
Unknown         1646

  state 'Unknown' count: 2,448
```

In [ ]:
```
# Convert Date Columns to Datetime
print("Before conversion:")
print(f"  start_date dtype: {df['start_date'].dtype}")
print(f"  Sample value:     {df['start_date'].iloc[0]}")

df['start_date'] = pd.to_datetime(df['start_date'], errors='coerce')

print(f"\nAfter conversion:")
print(f"  start_date dtype: {df['start_date'].dtype}")
print(f"  Date range: {df['start_date'].min().date()} to {df['start_date'].max().date()}")
print(f"  Total days covered: {(df['start_date'].max() - df['start_date'].min()).days} days")
```

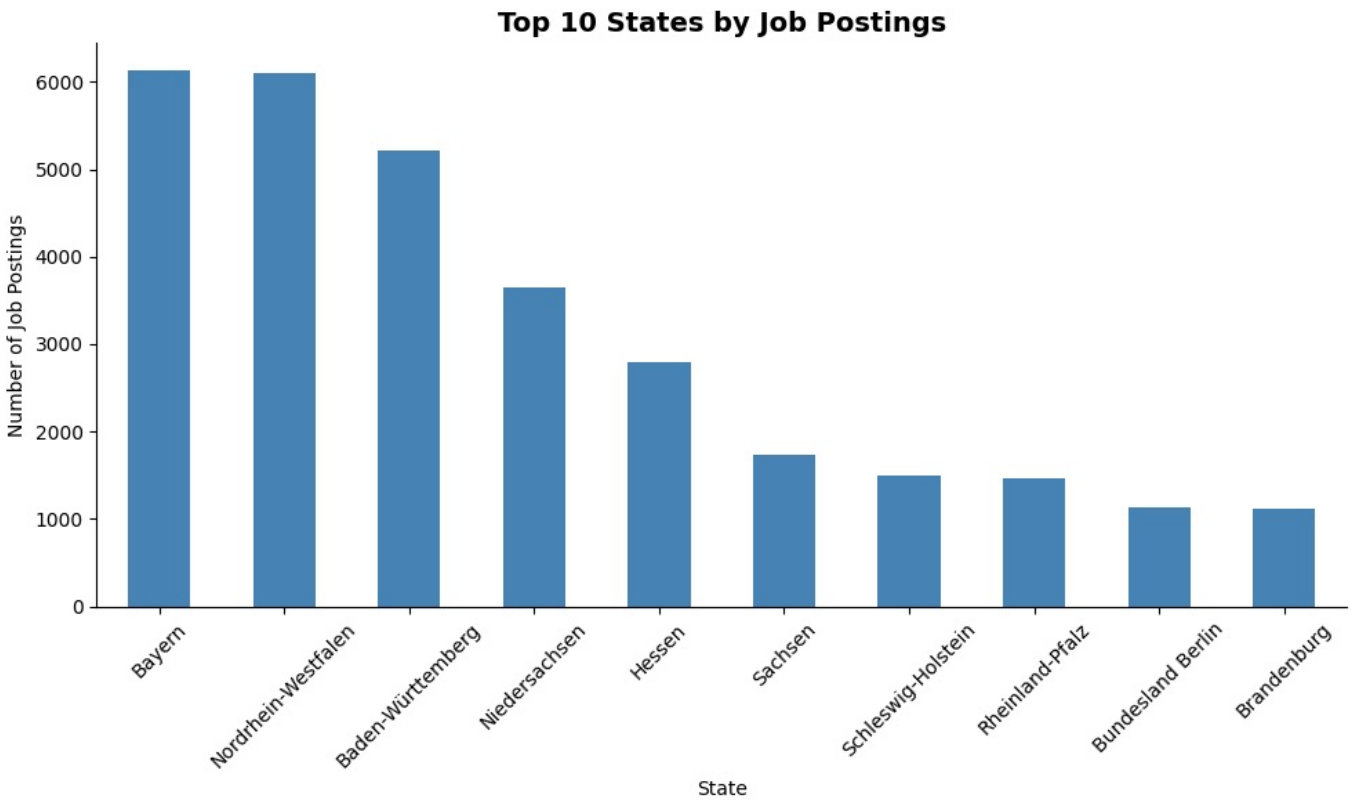
```
Before conversion:
  start_date dtype: object
  Sample value:     2026-03-02T23:00:00.000Z

After conversion:
  start_date dtype: datetime64[ns, UTC]
  Date range: 2026-02-11 to 2026-03-13
  Total days covered: 30 days
```

SECTION 4: Explanatory Data Visualisations

Here we will explain how we visualize the cleaned dataset to answer the seven questions outlined in the introduction. Each subsection focuses on one question with a single clear chart and a written insight.

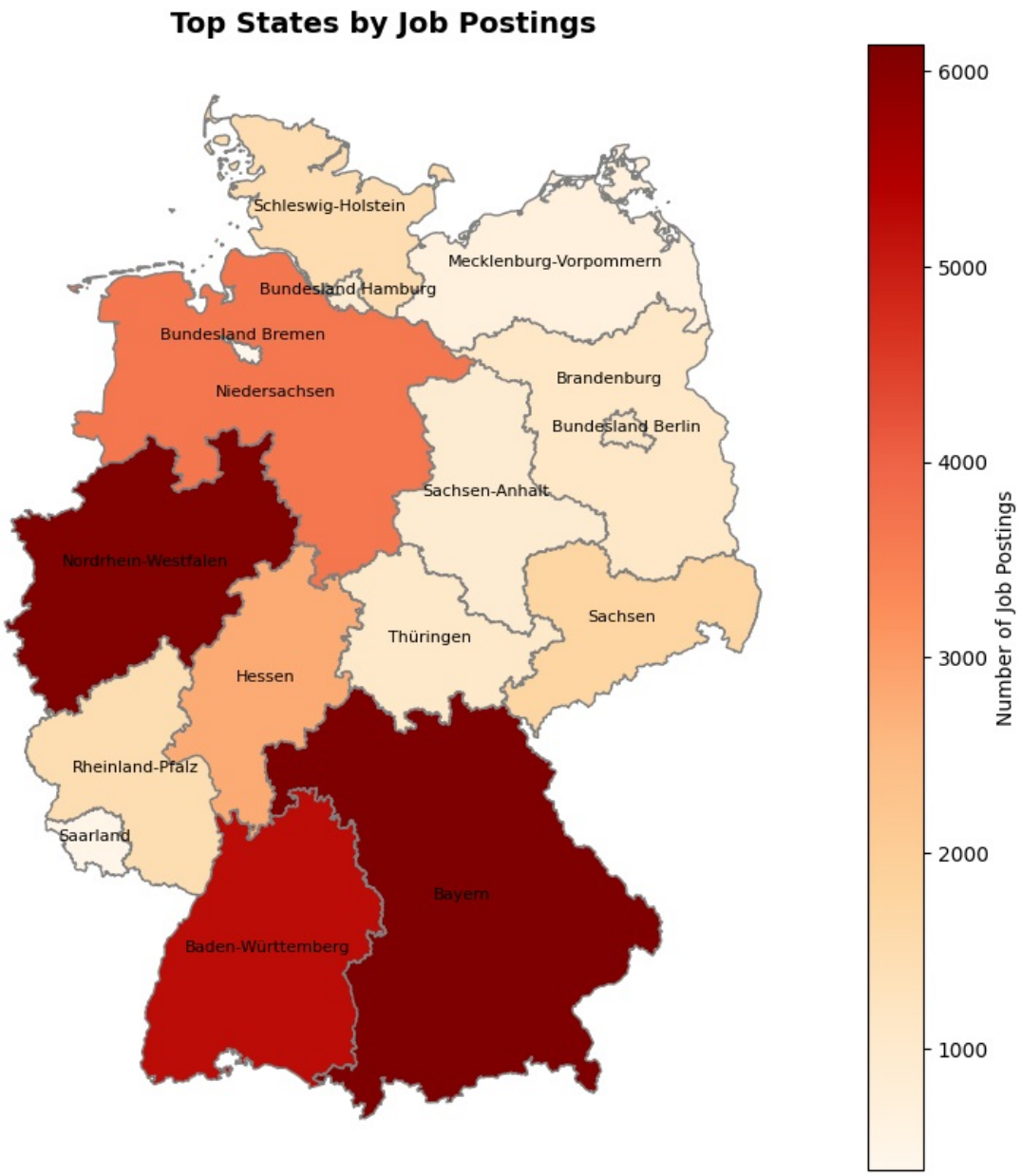
1. Where are jobs concentrated? (by state)

In [ ]:
```
state_counts = df[df['state'] != 'Unknown']['state'].value_counts().head(10)

plt.figure(figsize=(10, 6))
state_counts.plot(kind='bar', color='steelblue')
plt.ylabel('Number of Job Postings')
plt.xlabel('State')
plt.title('Top 10 States by Job Postings', fontsize=14, fontweight='bold')

plt.xticks(rotation=45)
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.tight_layout()
plt.show()
```



In [ ]:
```
import geopandas as gpd

geo_data_path = '/content/gadm41_DEU_1.json'
states = gpd.read_file(geo_data_path)
name_subs = {
    'Berlin': 'Bundesland Berlin',
    'Bremen': 'Bundesland Bremen',
    'Hamburg': 'Bundesland Hamburg',
}
states['NAME_1'] = states['NAME_1'].replace(name_subs)

state_counts = df[df['state'] != 'Unknown']['state'].value_counts()
states = states.merge(state_counts, left_on='NAME_1', right_on='state')
states.plot(column='count',
            cmap='OrRd',
            legend=True,
            legend_kwds={'label': 'Number of Job Postings', 'orientation': 'vertical'},
            figsize=(12, 10),
            edgecolor='gray')
# Name the states on the plot
brandenburg_offset = (-0.2, 0.4)
for i, row in states.iterrows():
  # Avoid overlap between Brandenburg and Bundesland Berlin
  if row['NAME_1'] == 'Brandenburg':
    plt.annotate(text=row['NAME_1'], xy=(row.geometry.centroid.x + brandenburg_offset[0], row.geometry.centroid
    continue
  plt.annotate(text=row['NAME_1'], xy=(row.geometry.centroid.x, row.geometry.centroid.y), horizontalalignment='
```

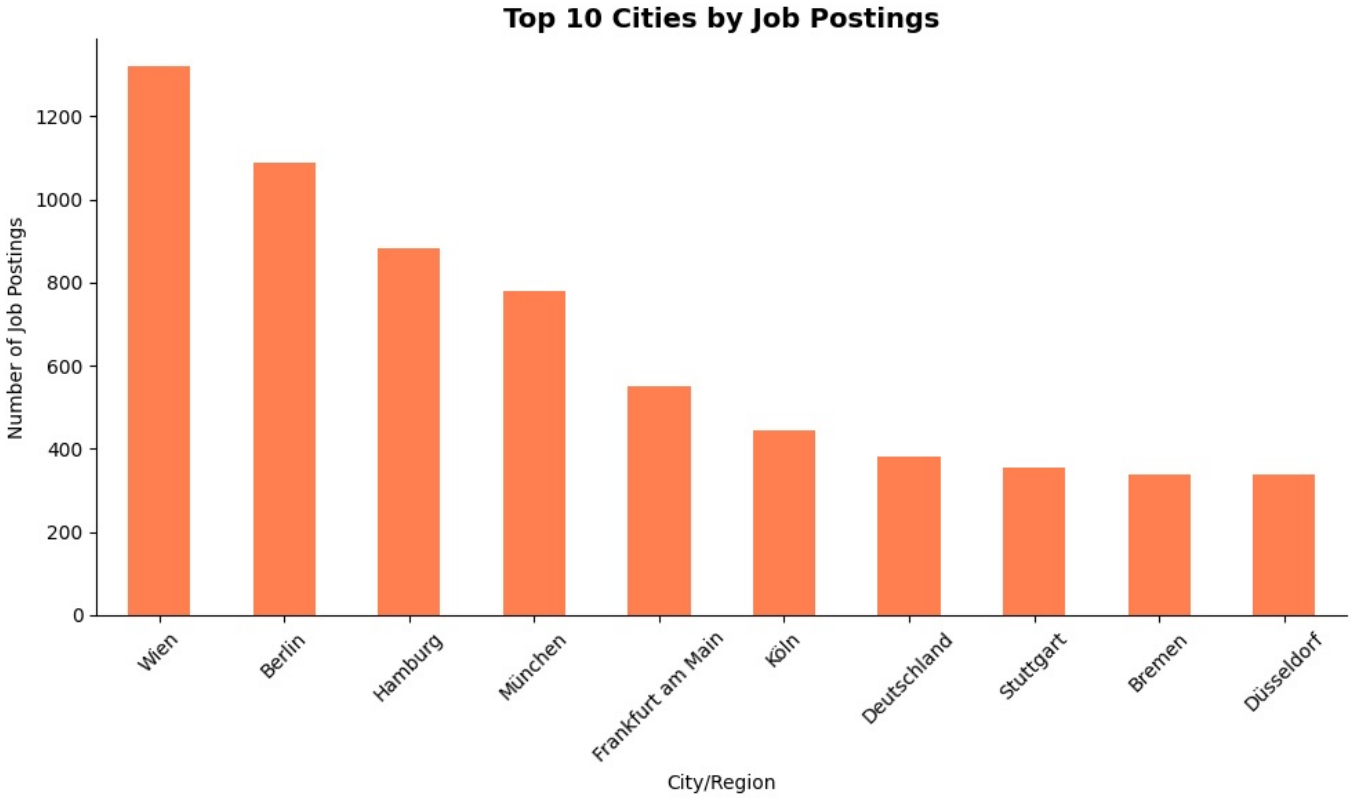
```
plt.title('Top States by Job Postings', fontsize=14, fontweight='bold')
plt.axis('off')
plt.show()
```



The top 3 states; Bayern, Nordrhein-Westfalen, and Baden-Württemberg are heavily concentrated by a large margin, accordingly accounting for a large share of all listings reflecting their roles as the major economic powerhouses underscoring strong manufacturing and services sectors in Germany. The abscence of the Eastern German states from the top 10 states suggests the platform is underpriveleged in those regions.

2. Which cities have the most listings?

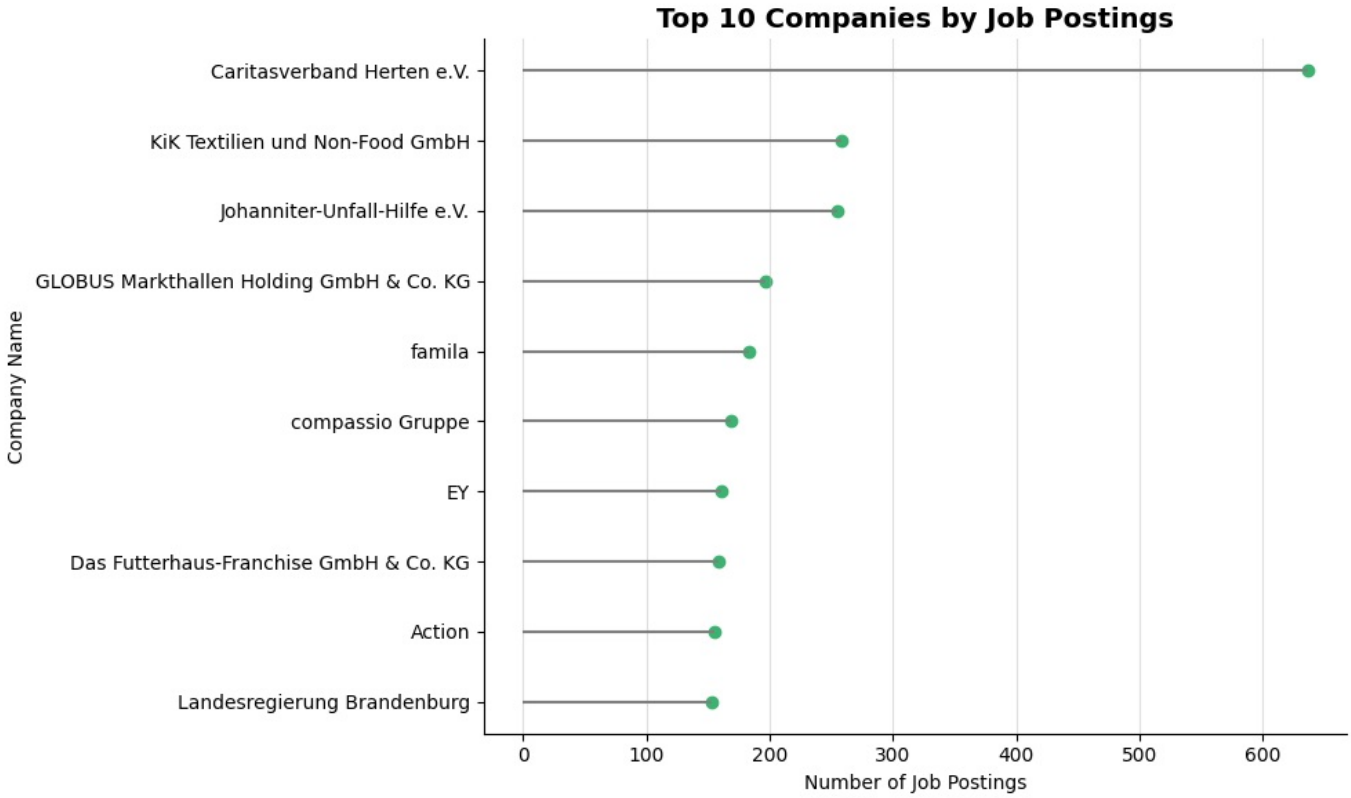
In [ ]:
```
city_counts = df['region'].value_counts().head(10)

plt.figure(figsize=(10, 6))
city_counts.plot(kind='bar', color='coral')
plt.ylabel('Number of Job Postings')
plt.xlabel('City/Region')
plt.title('Top 10 Cities by Job Postings', fontsize=14, fontweight='bold')

plt.xticks(rotation=45)
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.tight_layout()
plt.show()
```



Wien; capital city of Austria, emerges as the leading city inspite of Austria having a much smaller market than German cities such as Berlin, Munchen, and Hamburg. Although Stuttgart being a major economic hub, ranks lower which indicates potential growth opportunities.

3. Which companies are hiring?

In [ ]:
```
top_companies = df['company_name'].value_counts().head(10)

plt.figure(figsize=(10, 6))
plt.scatter(top_companies.values, top_companies.index, color='mediumseagreen')
plt.hlines(top_companies.index, 0, top_companies.values, color='gray')
plt.xlabel('Number of Job Postings')
plt.ylabel('Company Name')
plt.title('Top 10 Companies by Job Postings', fontsize=14, fontweight='bold')
plt.grid(axis="x", alpha=0.4)

plt.gca().invert_yaxis()
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

top5_share = top_companies.head(5).sum() / len(df) * 100
print(f"\nTop 5 companies account for {top5_share:.1f}% of all postings.")
print(f"Total unique companies: {df['company_name'].nunique():,}")
```




```
Top 5 companies account for 3.7% of all postings.
Total unique companies: 13,665
```

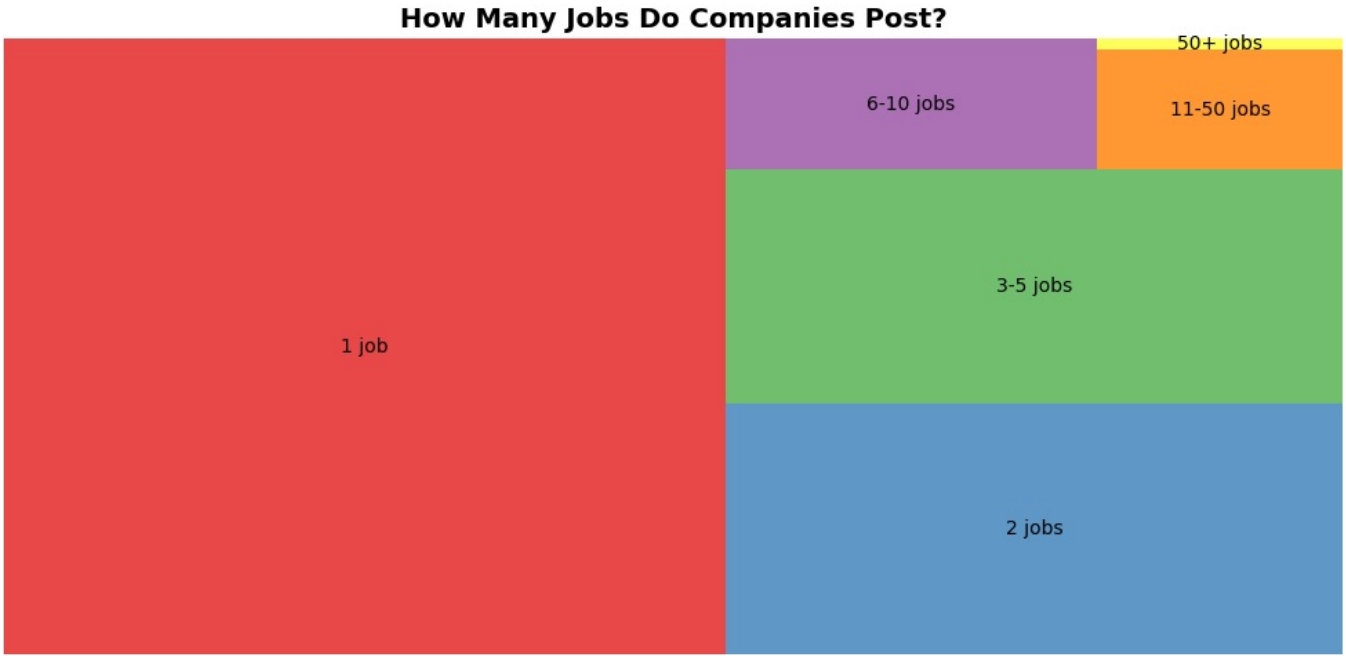
The leading employer from the Caritasverband Herten company, has a high volume of postings than the others. However, the Kik Textilien and Non-food GmbH to Compassio Gruppe company employers represent a small share of total listings. With over 13,000 unique companies, the platform demonstrates a high level of employer diversity which include social care, retail, and engineering firms reflecting the breadth of the Mittelstand.

4. How concentrated is the market?

In [ ]:

```python
company_counts = df['company_name'].value_counts()
# print(company_counts)

buckets = pd.cut(company_counts, bins=[0, 1, 2, 5, 10, 50, 1000],
                 labels=['1 job', '2 jobs', '3-5 jobs', '6-10 jobs', '11-50 jobs', '50+ jobs'])
print(buckets)
bucket_counts = buckets.value_counts().sort_index()

plt.figure(figsize=(10, 5))
squarify.plot(sizes=bucket_counts, label=bucket_counts.index, alpha=0.8, color=plt.cm.Set1.colors)
plt.title('How Many Jobs Do Companies Post?', fontsize=14, fontweight='bold')
plt.axis('off')

plt.tight_layout()
plt.show()

single = (company_counts == 1).sum()
print(f"Companies with only 1 posting: {single:,} ({single/len(company_counts)*100:.1f}%)")
print(f"Median postings per company: {company_counts.median():.0f}")
```

```
company_name
Caritasverband Herten e.V.                                  50+ jobs
KiK Textilien und Non-Food GmbH                             50+ jobs
Johanniter-Unfall-Hilfe e.V.                                50+ jobs
GLOBUS Markthallen Holding GmbH & Co. KG                    50+ jobs
famila                                                      50+ jobs
                                                              ...
HBI Europe GmbH                                               1 job
TeamCon GmbH                                                  1 job
REICHEL Ingenieurgesellschaft für Projektmanagement mbH       1 job
Rudolf Steiner Schule Altona e.V.                             1 job
Wigger Fenster + Fassaden GmbH                                1 job
Name: count, Length: 13665, dtype: category
Categories (6, object): ['1 job' < '2 jobs' < '3-5 jobs' < '6-10 jobs' < '11-50 jobs' < '50+ jobs']
```



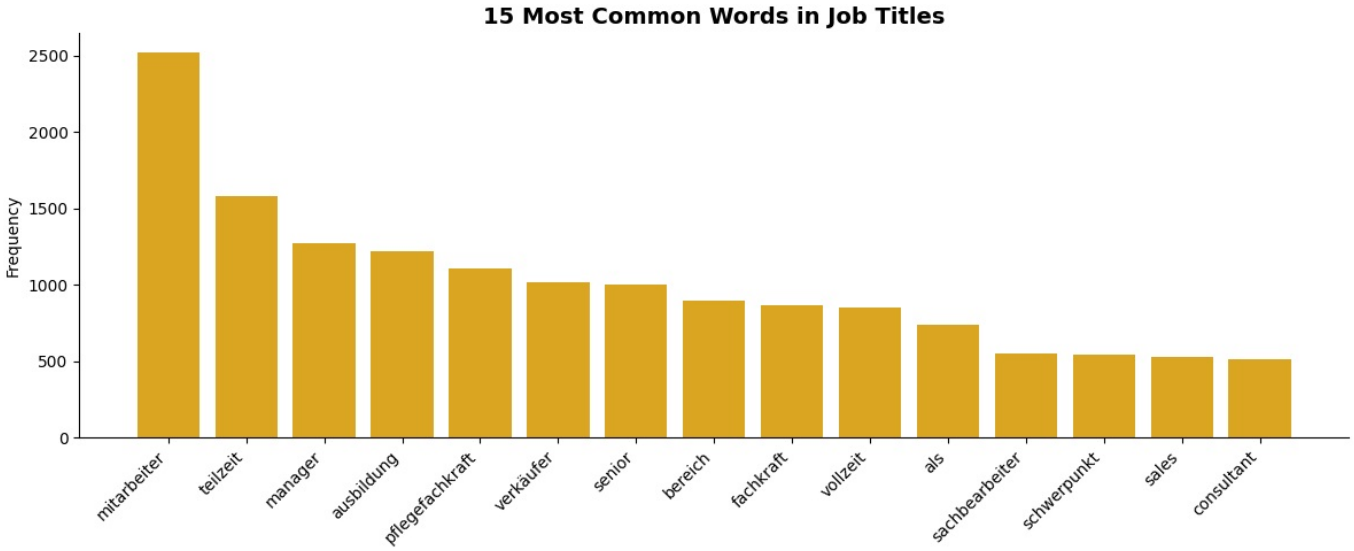

```
Companies with only 1 posting: 7,370 (53.9%)
Median postings per company: 1
```

Most companies post only 1 or 2 jobs indicating a classic long-tail distribution for SME platforms, where many small employers contribute limited domination rather than a few large firms play the role. This is beneficial for the platform, since the platform doesn't rely on any single client reducing the risk of the overall content.

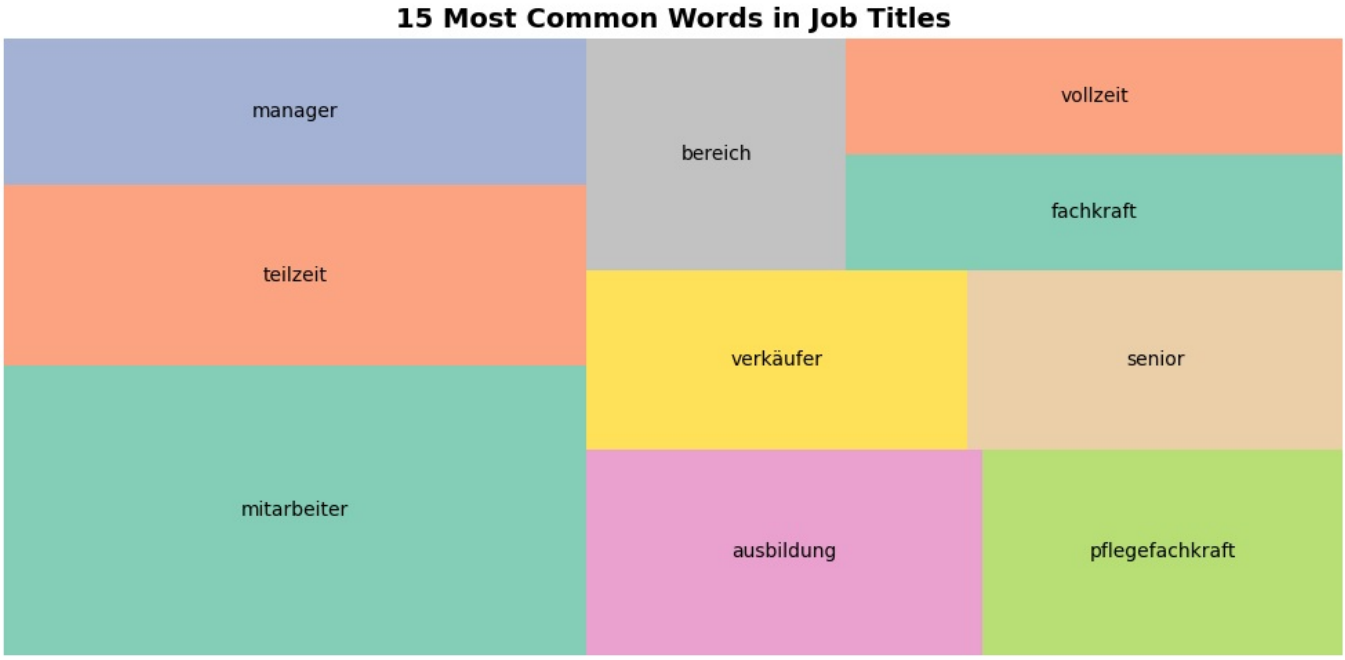
5. Which job categories are in demand?

In [ ]:

```
stopwords = {'m', 'w', 'd', 'und', 'in', 'im', 'für', 'der', 'die', 'das',
             'oder', 'von', 'mit', 'den', 'des', 'am', 'ein', 'eine', 'zur',
             'zum', 'bei', 'gmbh', 'co', 'kg', 'all', 'genders', 'mwd', 'mfd'}

words = []
for title in df['title'].dropna():
    tokens = re.findall(r'[A-Za-zÄÖÜäöüß]+', title.lower())
    words.extend([t for t in tokens if len(t) > 2 and t not in stopwords])

word_counts = Counter(words)
top_words = word_counts.most_common(15)
labels, counts = zip(*top_words)

plt.figure(figsize=(12, 5))
plt.bar(labels, counts, color='goldenrod')
plt.ylabel('Frequency')
plt.title('15 Most Common Words in Job Titles', fontsize=14, fontweight='bold')
plt.xticks(rotation=45, ha='right')

plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.tight_layout()
plt.show()
```



In [ ]:
```
stopwords = {'m', 'w', 'd', 'und', 'in', 'im', 'für', 'der', 'die', 'das',
             'oder', 'von', 'mit', 'den', 'des', 'am', 'ein', 'eine', 'zur',
             'zum', 'bei', 'gmbh', 'co', 'kg', 'all', 'genders', 'mwd', 'mfd'}

words = []
for title in df['title'].dropna():
    tokens = re.findall(r'[A-Za-zÄÖÜäöüß]+', title.lower())
    words.extend([t for t in tokens if len(t) > 2 and t not in stopwords])

word_counts = Counter(words)
top_words = word_counts.most_common(10)
labels, counts = zip(*top_words)

plt.figure(figsize=(10, 5))
squarify.plot(sizes=counts,
              label=labels,
              alpha=0.8,
              color=plt.cm.Set2.colors)
plt.title('15 Most Common Words in Job Titles', fontsize=14, fontweight='bold')
plt.axis("off")

plt.tight_layout()
plt.show()
```

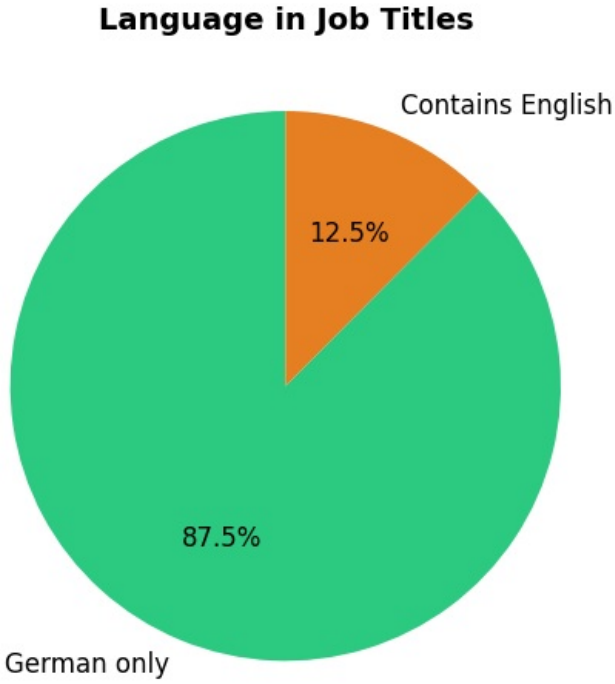


As indicated in the graph, Mitarbeiter(employee) is the most common term, showing strong demand for general operational roles. Teilzeit and Vollzeit also appear frequently as employment type is often specified directly in German job titles. Technical terms highlight like (Techniker, Mechaniker, and Ingenieur) the strong engineering and manufacturing orientation of the Mittelstand.

6. How is the market globally recognized?

In [ ]:
```
english_keywords = ['Manager', 'Engineer', 'Developer', 'Analyst', 'Lead', 'Senior',
                    'Junior', 'Specialist', 'Consultant', 'Controller', 'Designer',
```

```
                    'Director', 'Head of', 'Sales', 'Marketing']

df['has_english'] = df['title'].str.contains('|'.join(english_keywords), case=False, na=False)

eng_pct = df['has_english'].mean() * 100
ger_pct = 100 - eng_pct

plt.figure(figsize=(7, 5))
plt.pie([ger_pct, eng_pct],
        labels=['German only', 'Contains English'],
        autopct='%1.1f%%',
        colors=['#2cc980', '#e67e22'],
        startangle=90,
        textprops={'fontsize': 12})
plt.title('Language in Job Titles', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.show()

print(f"Jobs with English keywords: {df['has_english'].sum():,} out of {len(df):,}")
```



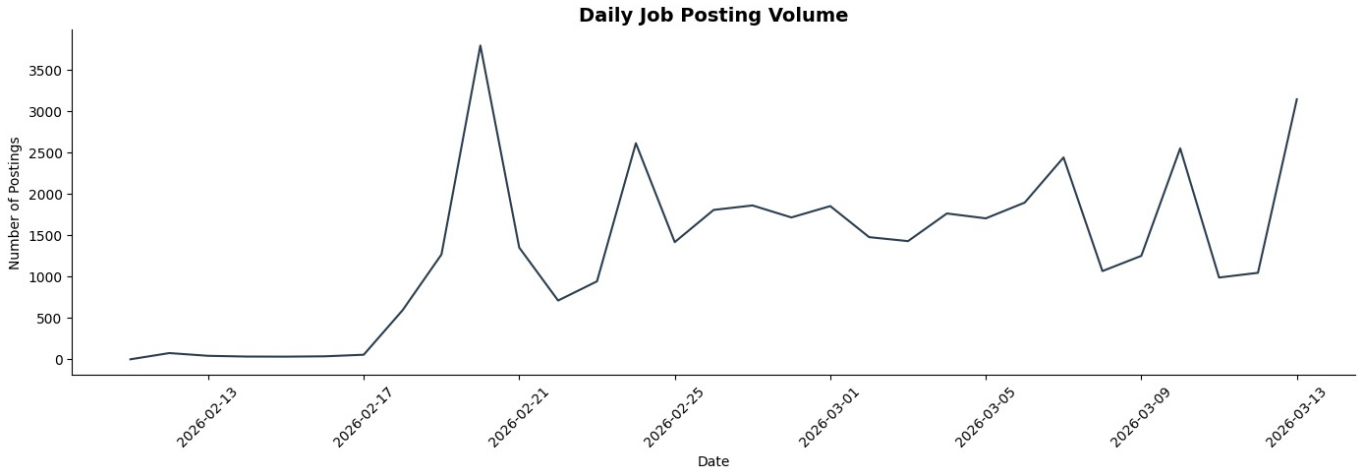

```
Jobs with English keywords: 5,108 out of 40,986
```

The majority of job titles are presented in German, indicating a primarly domestic market. However, the presence of English terms(example; Manager, Engineer, Developer) suggest that while Mittelstand remains largely local in focus, certain sectors specially in technology, finance, and management operate international terminology.

Overall, the expanding bilingual functionality could help to enhance both German and English-speaking job seekers.

7. When do recruiters post?

In [ ]:
```
daily = df.groupby(df['start_date'].dt.date).size().reset_index(name='count')
daily.columns = ['date', 'count']
daily['date'] = pd.to_datetime(daily['date'])

plt.figure(figsize=(14, 5))

plt.plot(daily['date'], daily['count'], color='#2c3e50', linewidth=1.5)
plt.xlabel('Date')
plt.ylabel('Number of Postings')
plt.title('Daily Job Posting Volume', fontsize=14, fontweight='bold')

plt.xticks(rotation=45)
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print(f"Peak day: {daily.loc[daily['count'].idxmax(), 'date'].strftime('%Y-%m-%d')} "
      f"with {daily['count'].max():,} postings")
print(f"Lowest day: {daily.loc[daily['count'].idxmin(), 'date'].strftime('%Y-%m-%d')} "
      f"with {daily['count'].min():,} postings")
```




```
Peak day: 2026-02-20 with 3,794 postings
Lowest day: 2026-02-11 with 2 postings
```

The line chart shows clear fluctuations in daily posting volume with noticeable declines on weekends when fewer jobs are posted. This indicates that hiring is concentrated on weekdays. Accordingly, the platform should enhance the engagement by timing email alerts and promotional activities during peak weekdays.

SECTION 5

Discussion and Conclusion

From our analysis which is based on Yourfirm.de dataset, we have tried to understand how this company plays a role in job opportunities for job seekers holding different information related to different job categories. We have explained about the seven meaningful insights and answered based on the visualization context. This analysis examined over 47,750 job postings from yourfirm dataset using Germany and Austria as an example.

Regarding job posting activities, jobs are posted more on weekdays suggesting structured recruitment styles. We have also found that job listings in Germany are heavily concentrated in the Western and Southern German regions(Bayern, NRW, Baden-Wurttemberg), while the Eastern part remains unprivileged in a high growth opportunity. The analysis indicates job markets related to manufacturing are written in German, which indicates the market is local and industry based though some jobs are written in English. However, the dataset shows certain limitations which include the abscence of salary information, numeric codes without labels, and restricted time frame of only one month which limits the depth of insights.

To build up the platform, we have made some recommendations such as expanding the economic outcomes in Eastern Germany which brings growth opportunities, improve bilingual search which helps both the Germans and global users, and add filters (like full-time or part-time) so that it helps the job seekers to search based on their employment types.